CYBERSECURITY PROJECT REPORT

THREAT INTELLIGENCE AGGREGATOR

A Python-based tool for collecting Indicators of Compromise (IoCs) from AlienVault OTX and enriching them with VirusTotal reputation data.

Name: P BOOMIKA

Department: Cyber Security

Institution: Sona College of Technology

Academic Year: 2026–2027

Project topic 12 — Threat Intelligence Aggregator

ABSTRACT

Cybersecurity teams use threat intelligence to understand indicators associated with suspicious or malicious activity. This project implements a command-line Threat Intelligence Aggregator in Python. It collects Indicators of Compromise (IoCs) from subscribed AlienVault Open Threat Exchange (OTX) pulses and can query VirusTotal for reputation and analysis statistics. The application normalizes indicator types, removes duplicates, handles API errors, and saves the results in JSON and CSV formats. An offline demo mode allows the application to be demonstrated without API keys or network requests. The tool is intended for defensive education and authorized analysis; results are advisory and must be verified.

1. INTRODUCTION

Threat intelligence helps security analysts contextualize IP addresses, domains, URLs, and file hashes that may be related to cyber threats. Reviewing multiple sources manually can take time and produce inconsistent records. A small aggregator can centralize collection and provide a repeatable workflow for recording and reviewing IoCs.

2. PROBLEM STATEMENT

Threat indicators are published across different services and formats. Analysts need a simple way to collect selected indicators, enrich them with reputation information, remove duplicates, and export results for further review. The project addresses this need with a lightweight Python CLI and provider APIs.

3. OBJECTIVES

- Collect IoCs from the user's subscribed OTX pulses.
- Accept manually supplied indicators for authorized investigation.
- Enrich supported indicators with VirusTotal reputation and analysis statistics.
- Deduplicate records and include timestamps and source details.
- Export results to machine-readable JSON and spreadsheet-friendly CSV.
- Provide a no-network demo mode and document safe, ethical use.

4. SCOPE AND LIMITATIONS

The project supports IPv4/IPv6 addresses, domains, URLs, and common MD5/SHA-1/SHA-256 hash formats for classification. It performs read-only API lookups; it does not scan target systems or execute suspicious files. Live results depend on provider availability, account permissions, quotas, and API response quality. Indicator classification is intentionally lightweight and is not a substitute for a dedicated validation library.

5. SYSTEM REQUIREMENTS

Hardware: a standard computer with internet access for live mode. Software: Python 3.10 or newer and the requests library. Live OTX collection requires an AlienVault OTX API key. VirusTotal enrichment requires a VirusTotal API key. The offline demo requires neither key.

6. SYSTEM DESIGN

The application follows a simple pipeline:

1. Parse command-line options and read API keys from environment variables.
2. Collect indicators from OTX subscribed pulses and/or command-line arguments.
3. Classify indicator types and remove duplicate values.
4. Query VirusTotal for supported indicators when a key is configured and the enrichment limit allows it.
5. Handle missing records, rate limits, HTTP errors, and network exceptions.
6. Save normalized records to JSON and CSV, then print a summary.

7. IMPLEMENTATION DETAILS

The source file main.py contains these principal functions: **classify_indicator()** identifies common IP, hash, URL, and domain formats; **fetch_otx_indicators()** requests subscribed pulse data from OTX; **enrich_virustotal()** queries the VirusTotal v3 API using read-only GET requests; **write_json()** and **write_csv()** create output reports; and **main()** validates arguments, coordinates collection, and displays the summary.

8. INSTALLATION AND EXECUTION

1. Install Python 3.10 or newer.
2. Create and activate a virtual environment.
3. Run `python -m pip install -r requirements.txt`.
4. For the first demonstration, run `python main.py --demo`.
5. For live OTX collection, set the OTX_API_KEY environment variable. Set VT_API_KEY if VirusTotal enrichment is required.
6. Run `python main.py --limit 5 --max-enrich 10`.
7. Review the generated threat_intel_report.json and threat_intel_report.csv files.

9. SAMPLE OUTPUT

```
======================================================================
THREAT INTELLIGENCE AGGREGATOR
Generated: <runtime timestamp>
Indicators collected: 4
JSON report: <current directory>/threat_intel_report.json
CSV report : <current directory>/threat_intel_report.csv
----------------------------------------------------------------------
01. 198.51.100.23 [IPv4]
    Source: DEMO DATA — NOT LIVE INTELLIGENCE
    VirusTotal: demo: no API request made
02. example.org [domain]
    Source: DEMO DATA — NOT LIVE INTELLIGENCE
    VirusTotal: demo: no API request made
...
======================================================================
Note: reputation data is advisory and can be incomplete or incorrect.
```

This is illustrative output from offline demo mode. Timestamps and output paths vary. Demo values are not claims of current malicious activity.

10. TESTING

Test 1 — Offline demo: Run python main.py --demo. Expected: successful execution and JSON/CSV reports with four clearly labelled demo records.
Test 2 — Custom indicator: Run python main.py --indicator example.org --max-enrich 0. Expected: a report is written without a VirusTotal request.
Test 3 — Argument validation: Run python main.py --limit 0. Expected: the CLI rejects the invalid limit.
Test 4 — Missing keys: Run without API keys. Expected: a warning for unavailable OTX collection and skipped VirusTotal enrichment.
Test 5 — Provider response: With authorized keys, verify that API outcomes are handled as status values such as ok, not found, or rate limited.

11. SECURITY, PRIVACY, AND ETHICAL CONSIDERATIONS

• Use only for defensive education and authorized investigations.
• The tool makes read-only API requests and does not probe or disrupt third-party systems.
• Never commit API keys to source control or share them in screenshots.
• Follow OTX and VirusTotal terms, privacy policies, and rate limits.
• Do not submit confidential or personal indicators to third-party services without permission.
• A reputation score is not proof. Validate findings using context and multiple trusted sources.
• Do not automatically block, accuse, or take action against an entity based only on this report.

12. FUTURE ENHANCEMENTS

Possible improvements include adding more threat-intelligence feeds, configurable caching, richer domain validation, scheduled collection, database storage, a dashboard, and signed export reports. Any expansion should preserve API-key protection, rate limiting, and privacy safeguards.

13. CONCLUSION

The Threat Intelligence Aggregator demonstrates how Python can integrate security APIs to collect, normalize, enrich, and export threat indicators. It provides a practical introduction to API usage, data handling, error management, and defensive security workflows. Its demo mode supports classroom demonstrations, while live mode can be used with valid API credentials and in accordance with provider policies. The output is intended to assist human analysis rather than replace it.

REFERENCES

1. AlienVault Open Threat Exchange (OTX): https://otx.alienvault.com/
2. VirusTotal: https://www.virustotal.com/
3. VirusTotal API v3 documentation: https://docs.virustotal.com/reference/overview
4. Python documentation: https://docs.python.org/3/
5. Requests documentation: https://requests.readthedocs.io/